Grafos, representación y operaciones

Algoritmos y Estructuras de Datos II
Ingeniería en Sistemas

Introducción a Grafos

Los grafos aparecen cuando necesitamos representar elementos que tienen algún tipo de relación entre sí. En informática esto sucede constantemente: ciudades conectadas por rutas, computadoras dentro de una red, usuarios en redes sociales o páginas web enlazadas entre sí.

La idea central de un grafo es bastante simple: representar objetos y las conexiones existentes entre ellos.

Grafos en la vida real

Aunque los vamos a estudiar dentro de Estructuras de Datos, los grafos también se utilizan en muchas otras áreas. En sociología sirven para analizar relaciones entre personas; en geografía permiten modelar mapas y caminos; en ingeniería eléctrica ayudan a representar redes; y en química se utilizan para describir estructuras moleculares.

En todos los casos aparece el mismo concepto: elementos conectados.

Grafos en informática

Dentro de la informática, los grafos se utilizan para resolver problemas donde existen relaciones o conexiones entre datos. Muchas aplicaciones que usamos todos los días funcionan internamente mediante estructuras basadas en grafos.

Por ejemplo, una aplicación de GPS representa ciudades y calles mediante un grafo para calcular rutas. Las redes sociales utilizan grafos para modelar usuarios y relaciones de amistad o seguimiento. Internet también puede verse como un enorme grafo donde las computadoras y routers están conectados entre sí.

Incluso los buscadores web trabajan con grafos: cada página es un nodo y cada enlace hacia otra página representa una conexión.

Temas de la unidad

En esta unidad vamos a trabajar con los conceptos fundamentales de los grafos y con las distintas formas de representarlos dentro de la memoria de la computadora.

Primero veremos las definiciones básicas necesarias para comprender qué es un grafo y cuáles son sus elementos principales. Luego estudiaremos las dos representaciones más utilizadas: la **matriz de adyacencia** y las **listas de adyacencia**.

Más adelante analizaremos cómo recorrer un grafo, cómo determinar conexiones entre vértices y cómo identificar componentes conexas. También veremos la matriz de caminos y el concepto de cierre transitivo, que permiten estudiar la alcanzabilidad dentro del grafo.

Finalmente trabajaremos con puntos de articulación, es decir, vértices cuya eliminación puede afectar la conectividad de toda la estructura.

¿Qué es un grafo?

Un grafo es una estructura que permite representar objetos y las relaciones que existen entre ellos.

Los objetos se representan mediante **vértices** (o nodos) y las relaciones mediante **arcos** (o aristas).

Formalmente, un grafo se representa mediante el par:

G = (V, A)

donde:

- **V** representa el conjunto de vértices.
- **A** representa el conjunto de arcos o aristas.

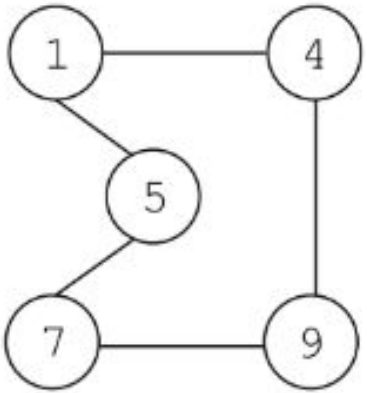
Un grafo puede utilizarse para modelar personas conectadas en una red social, ciudades unidas por rutas o computadoras dentro de una red.

Vértices y aristas

Los vértices representan los elementos que forman parte del problema que queremos modelar.

Las aristas representan las relaciones o conexiones existentes entre esos elementos.

Por ejemplo, si se desea representar un mapa de ciudades, cada ciudad puede modelarse mediante un vértice y cada ruta mediante una arista.

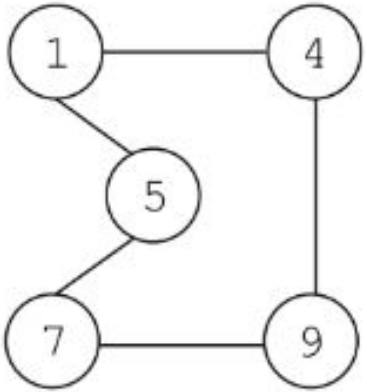


"La imagen permite identificar claramente los vértices y las conexiones existentes entre ellos."

Grafo no dirigido

En un grafo no dirigido las conexiones no tienen dirección.

Cuando existe una relación entre dos vértices, ambos pueden alcanzarse mutuamente a través de la misma arista.



Por esta razón, las aristas se representan mediante líneas simples que unen dos nodos.

Por ejemplo, si dos ciudades están conectadas por una ruta de doble sentido, la relación puede modelarse mediante un grafo no dirigido.

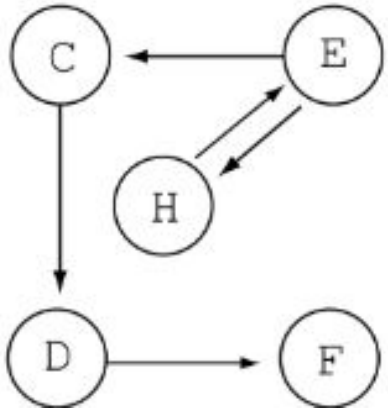
Grafo dirigido

Existen situaciones donde la relación tiene un sentido determinado.

En estos casos se utilizan **grafos dirigidos**, también llamados **dígrafos**.

Las conexiones se representan mediante flechas que indican la dirección de la relación.

Si existe un arco desde un vértice **u** hacia un vértice **v**, no necesariamente existe la relación inversa.



Un ejemplo típico son las redes sociales donde un usuario puede seguir a otro sin que la relación sea recíproca.

Grafos Dirigidos

Se utilizan cuando la relación tiene un sentido específico.



Caso 1- Instagram/X : Juan sigue a Messi, pero no necesariamente Messi sigue a Juan

Caso 2- Web : Google utiliza este tipo de relacion para determinar la importancia de cada pagina

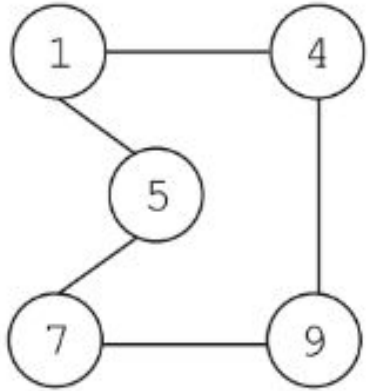


Adyacencia

Dos vértices se consideran **adyacentes** cuando existe una arista que los conecta.

En el grafo de la figura:

- 1 es adyacente a 4.
- 1 es adyacente a 5.
- 7 es adyacente a 9.



La adyacencia es uno de los conceptos más importantes porque será la base para representar grafos mediante matrices y listas de adyacencia.

Adyacencia

Es uno de los conceptos más utilizados en los algoritmos de grafos porque prácticamente todas las operaciones parten de responder la pregunta:

Caso 1 - Red social: Si Juan quiere ver las publicaciones de sus amigos directos, el sistema consulta los vértices adyacentes.



Caso 2 - GPS: Cuando una aplicación calcula una ruta, desde una calle necesita saber a qué calles puede pasar directamente.



Grafos valorados

En algunos problemas no alcanza con saber que dos vértices están conectados.

También es necesario almacenar información asociada a la conexión.

Esa información recibe el nombre de **peso** o **factor de peso**.

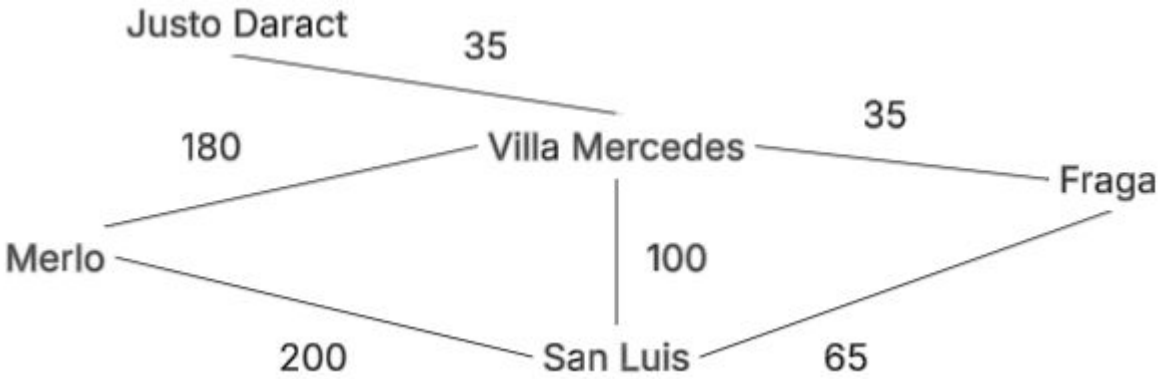
El peso puede representar distintas magnitudes según el problema:

- distancia entre ciudades,
- tiempo de viaje,
- costo económico,
- cantidad de datos transmitidos,
- capacidad de una conexión.

Grafos Valorados

Supongamos un conjunto de ciudades unidas por rutas.

Además de conocer qué ciudades están conectadas, interesa saber cuántos kilómetros separan a cada una.



En ese caso, cada arista almacena un valor numérico que representa la distancia.

A este tipo de estructura se la denomina **grafo valorado** o **grafo ponderado**.

Grafos Valorados

Se utilizan cuando la conexión tiene asociada una cantidad numérica.

La conexión existe, pero además tiene un costo, una distancia o una medida asociada.

Tiempo de Viaje: El peso representa el tiempo de Viaje. Google Maps trabaja con este tipo de Grafos.



Algoritmo famoso:

- Dijkstra

Ruta crítica (PERT/CPM)

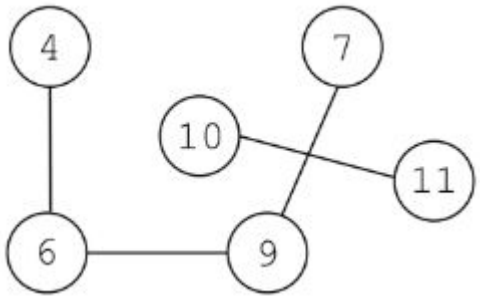
- El objetivo es encontrar el camino crítico del proyecto.

Grado de un vértice

Una característica importante de un vértice es su **grado**, que indica cuántas conexiones posee dentro del grafo.

En un grafo no dirigido, el grado de un vértice es simplemente la cantidad de aristas que inciden sobre él.

Cuanto mayor sea el grado de un vértice, mayor será la cantidad de conexiones directas que posee con otros nodos.



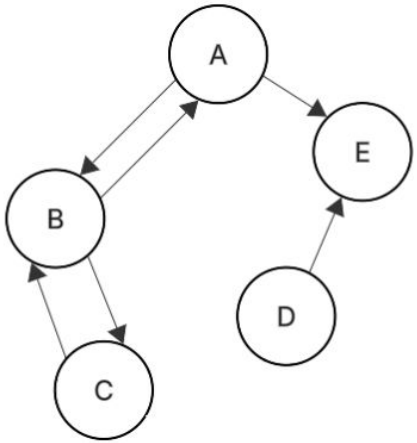
- grado(4) = 1
- grado(6) = 2
- grado(7) = 1
- grado(10) = 1
- grado(11) = 1
- grado(9) = 2

Grado de entrada y grado de salida

En los grafos dirigidos las conexiones tienen dirección, por lo que se distinguen dos conceptos.

El **grado de entrada** indica cuántas flechas llegan a un vértice.

El **grado de salida** indica cuántas flechas salen de un vértice.

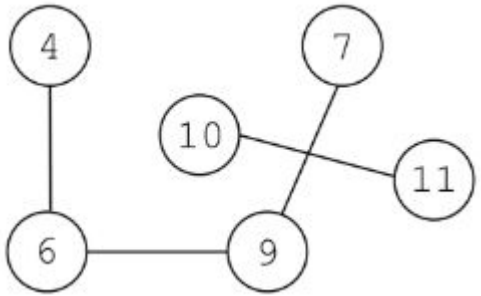


- grado ent.(A) = 1
- grado sal.(A) = 2
- grado ent.(D) = 0
- grado sal.(D) = 1

Concepto de camino

Un camino es una secuencia de vértices conectados mediante aristas.

Permite desplazarse desde un vértice inicial hasta otro vértice del grafo siguiendo conexiones válidas.



Ejemplos de caminos:

(4, 6, 9, 7)

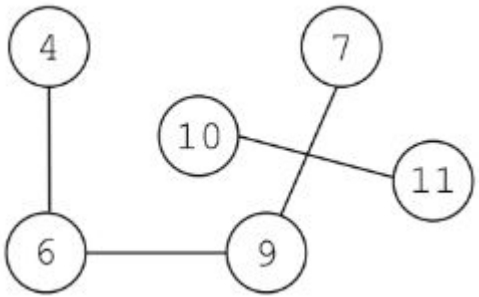
(10, 11)

Ambos representan recorridos válidos dentro del grafo.

Longitud de un camino

La longitud de un camino está determinada por la cantidad de aristas que lo forman.

No importa cuántos vértices aparecen, sino cuántas conexiones se recorren.



Ejemplo:

(4, 6, 9, 7)

Tiene longitud 3.

$4 \rightarrow 6$
$6 \rightarrow 9$
$9 \rightarrow 7$

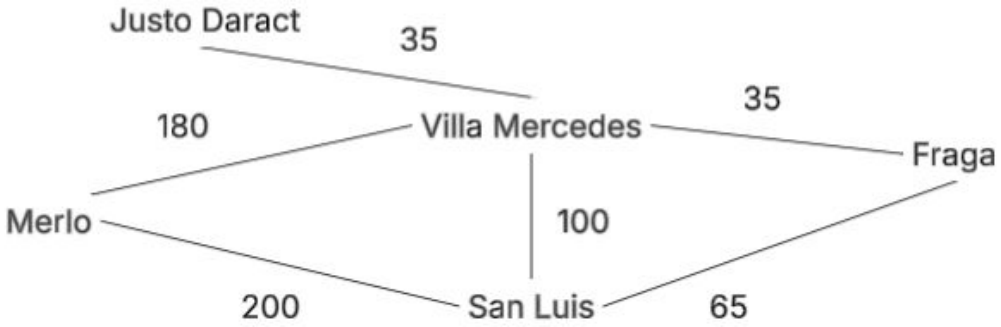
Tres aristas.

Longitud de un camino

En grafos valorados

La longitud se obtiene sumando los pesos de las aristas.

Por ejemplo:



Fraga – 35 – V. Mercedes – 100 – San Luis

Longitud Total: 135

Relación con Inv. Operativa

Cuando se busca el camino más corto entre dos ciudades, normalmente se utiliza la suma de los pesos.

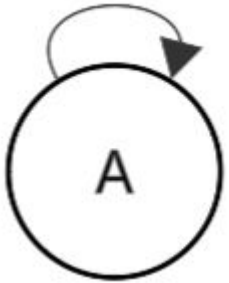
Los pesos pueden representar:

- kilómetros,
- tiempo,
- costo,
- consumo de combustible.

Bucle

En algunos grafos puede existir una arista que conecta un vértice consigo mismo.

A esta situación se la denomina **bucle**.



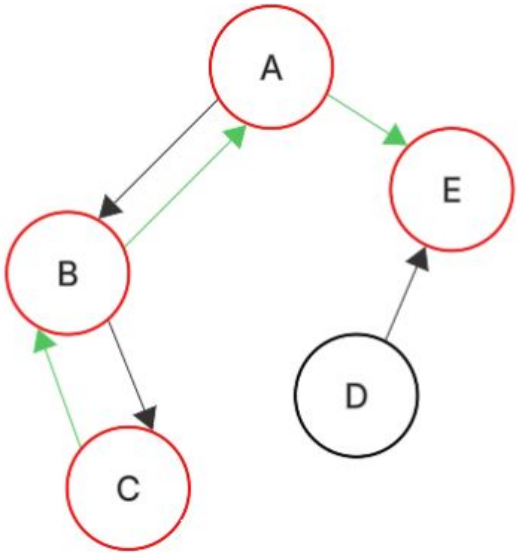
El vértice A posee una relación consigo mismo.

Los bucles no son muy frecuentes en la mayoría de los problemas prácticos, aunque existen aplicaciones específicas donde aparecen.

Camino simple

Un camino es simple cuando no repite vértices durante el recorrido.

La única excepción permitida es que el primer y último vértice coincidan cuando se está formando un ciclo.



Camino simple:

(C, B, A, E)

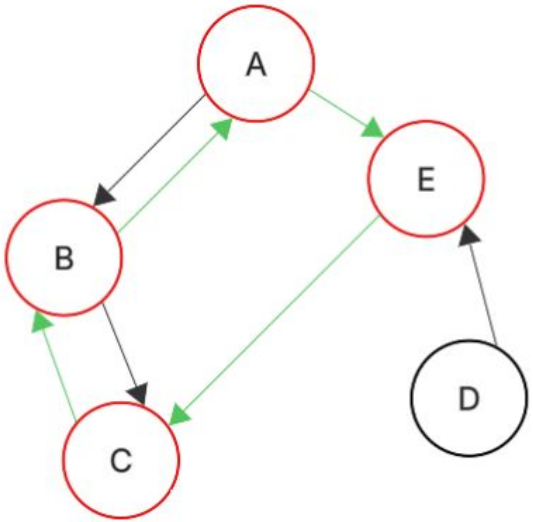
No es simple:

(C, B, A, B)

Ciclos

Un ciclo es un camino que comienza y termina en el mismo vértice.

Para ser considerado ciclo debe recorrer al menos una vuelta completa por el grafo.



En la figura puede observarse el ciclo:

(C, B, A, E, C)

La longitud del ciclo es 4.

Ciclos

¿Por qué importan los ciclos?

Muchos algoritmos deben detectar ciclos para evitar recorridos infinitos.

Además, algunos problemas requieren trabajar con grafos que no tengan ciclos.

Porque pueden generar recorridos infinitos.

Ejemplo real: GPS

Imaginemos calles:



Si el algoritmo no detectara ciclos podría quedarse recorriendo siempre el mismo circuito.

¿Por qué a veces NO queremos ciclos?

Porque representan dependencias imposibles.

Ejemplo: materias universitarias



Ahora imaginemos:



Esto implicaría:

- para cursar Prog II necesito Prog I
- para cursar Prog I necesito Prog II

Por eso los sistemas de correlatividades suelen modelarse con DAGs.

Grafos dirigidos acíclicos (DAG)

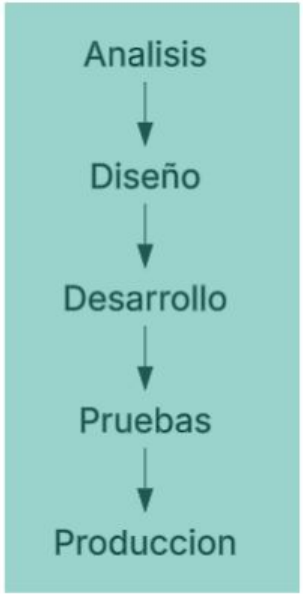
Un grafo dirigido acíclico es un grafo dirigido que no contiene ciclos.

También se conoce como:

DAG (Directed Acyclic Graph)

Ejemplos de uso

- ❏ Dependencias de software.
- ❏ Planificación de proyectos.
- ❏ Árboles de compilación.
- ❏ Flujo de tareas.

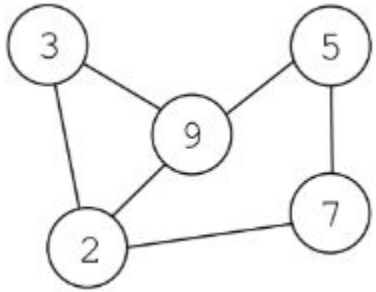


No es posible volver hacia atrás.

Grafos conexos

Un grafo es conexo cuando existe un camino entre cualquier par de vértices.

Dicho de otra manera, todos los nodos pertenecen a una misma red de conexiones.

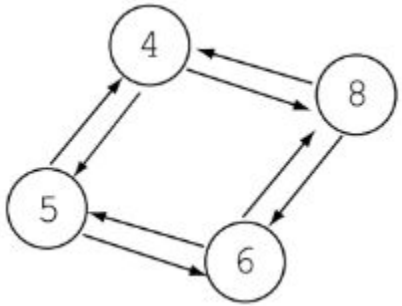


Desde cualquier vértice puede llegarse a cualquier otro recorriendo alguna secuencia de aristas.

Grafos fuertemente conexos

En un grafo dirigido la conectividad es más exigente.

Se dice que un grafo es fuertemente conexo cuando es posible llegar desde cualquier vértice a cualquier otro siguiendo la dirección de las flechas.



La dirección de los arcos permite desplazarse entre todos los vértices en ambos sentidos mediante distintos caminos.

El TAD Grafo

Hasta ahora estudiamos qué es un grafo desde el punto de vista conceptual.

Sin embargo, en programación no alcanza con definir los elementos de una estructura. También es necesario definir qué operaciones pueden realizarse sobre ella.

Por este motivo se define el **Tipo Abstracto de Datos (TAD) Grafo**.

Un TAD describe las operaciones disponibles sobre una estructura sin preocuparse todavía por cómo serán implementadas.

Operaciones básicas de un grafo

Todo Grafo debe permitir crear, modificar y consultar vértices y aristas.

Las operaciones más habituales son:

- nuevoVertice(u)
- arista(u,v)
- aristaPeso(u,v,w)
- borraArco(u,v)
- borraVertice(u)
- adyacente(u,v)

Estas operaciones existirán independientemente de cómo se almacene el grafo en memoria.

Ejemplo práctico

Supongamos que estamos modelando una red social.

Inicialmente: Ana / Juan

nuevoVertice("Ana");
nuevoVertice("Juan"); ***/ creamos dos vértices.***

arista("Ana","Juan"); ***/ representa una relación entre ambos usuarios.***

Visualmente:

Ana -------- Juan

Operación adyacente()

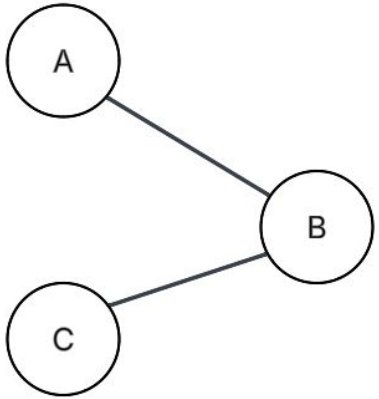
Una de las consultas más frecuentes consiste en verificar si dos vértices están conectados.

Por ejemplo:

adyacente("Ana","Juan") / **TRUE si existe relación entre ellos**
or (FALSE) si no existe relación entre ellos

Del dibujo a la memoria

Hasta ahora los grafos fueron representados mediante diagramas.



Este dibujo es útil para comprender el problema.

Pero una computadora no almacena dibujos.

Necesitamos una representación interna que permita almacenar vértices y conexiones de forma eficiente.

Cómo representar un grafo?

Existen dos estrategias clásicas para almacenar grafos en memoria.

- La primera utiliza una estructura secuencial basada en matrices.
- La segunda utiliza estructuras dinámicas enlazadas.

Dependiendo de las características del problema, una puede resultar más conveniente que la otra.

Cuál representación conviene utilizar?

La elección de una representación no es arbitraria. En informática siempre existe un compromiso entre el tiempo de ejecución de las operaciones y la cantidad de memoria utilizada.

Por esa razón, la representación elegida dependerá principalmente de:

1. la cantidad de vértices,
2. la cantidad de aristas,
3. las operaciones que se realizarán con mayor frecuencia.

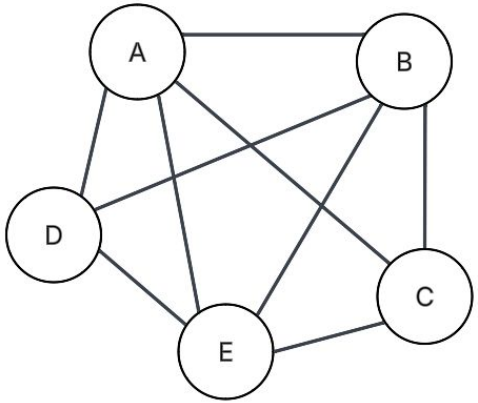
Un mismo grafo puede representarse mediante una **matriz de adyacencia** o mediante **listas de adyacencia**, pero el costo computacional de las operaciones será diferente.

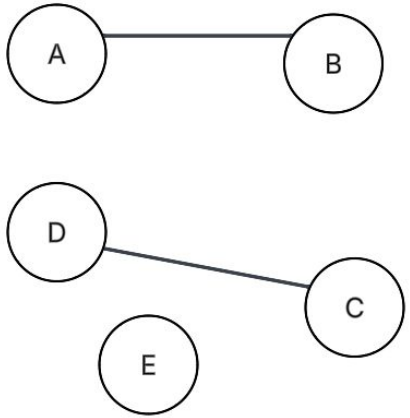
Grafos densos y grafos dispersos

Antes de elegir una representación es importante observar cuántas conexiones posee el grafo.

Un grafo se considera **denso** cuando tiene una gran cantidad de las conexiones posibles.

Por el contrario, se considera **disperso** cuando posee relativamente pocas conexiones respecto del máximo posible.





Pensando en memoria

Supongamos un grafo con:

1000 vértices

Una matriz de adyacencia necesitará:

1000 × 1000 = 1.000.000 posiciones

aunque solamente existan unas pocas conexiones.

Por lo tanto, una gran parte de la matriz contendrá ceros.

Si solamente existen 20 aristas:

Tiene sentido reservar espacio para un millón de posiciones?

Pensando en tiempo de ejecución

Ahora imaginemos la siguiente consulta:

¿Existe una conexión directa entre A y B?

Con una matriz de adyacencia basta acceder a una posición:

matriz[A][B]

La respuesta se obtiene inmediatamente.

Desde el punto de vista de complejidad:

O(1)

Tiempo constante.

El compromiso clásico

En estructuras de datos suele aparecer el siguiente compromiso:

- Más memoria → consultas más rápidas
- Menos memoria → consultas más lentas

Matriz de adyacencia

- Consume más memoria.
- Permite verificar adyacencia muy rápidamente.

Lista de adyacencia

- Consume menos memoria.
- Algunas búsquedas requieren recorrer listas.

Matriz de adyacencia

La matriz de adyacencia es la representación más sencilla de entender.

Si el grafo posee n vértices, se construye una matriz cuadrada de **n × n** posiciones.

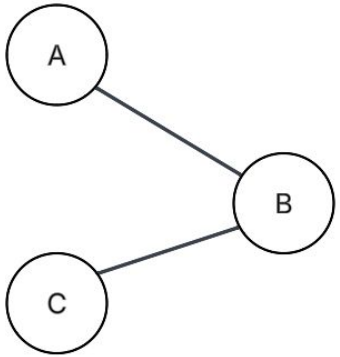
Cada fila y cada columna representan un vértice.

El contenido de cada celda indica si existe o no una conexión entre ambos vértices.

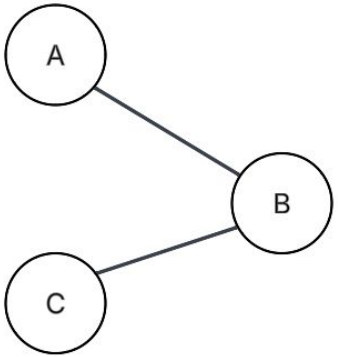
Idea básica:

Para cada par de vértices

1 → existe una arista
0 → no existe una arista



Construcción de la matriz



Las conexiones son:

(A,B)
(A,C)

La matriz resultante será:

	A	B	C
A	0	1	0
B	1	0	1
C	0	1	0

En los grafos no dirigidos la matriz de adyacencia siempre es simétrica ya que las relaciones entre vértices no son ordenadas:

Ventajas y desventajas

Ventajas

La consulta de adyacencia es extremadamente rápida.

¿A está conectado con C?

matriz[A][B]

Complejidad: **O(1)**

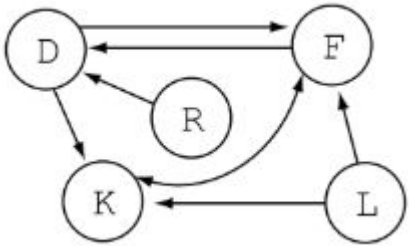
Desventajas

La memoria utilizada crece con: **$O(n^2)$**

Por lo tanto, cuando existen muchos vértices y pocas conexiones, gran parte de la matriz queda desaprovechada.

Construcción de la matriz

Ahora bien, supongamos un grafo dirigido como el de la figura, el cual posee un orden dado {D,F,K,L,R},



entonces la matriz de adyacencia:

$$\mathbf{A} = \begin{vmatrix} 0 & 1 & 1 & 0 & 0 \\ 1 & 0 & 1 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 0 & 0 \\ 1 & 0 & 0 & 0 & 0 \end{vmatrix}$$

Matriz de pesos

Hasta ahora vimos que la matriz de adyacencia solamente indica si existe o no una conexión entre dos vértices.

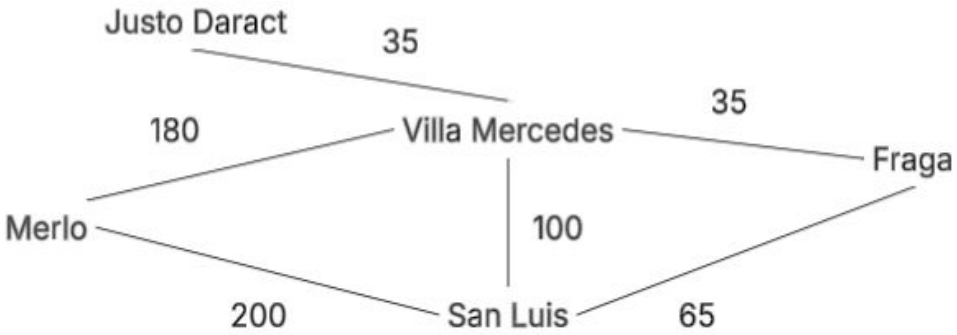
Sin embargo, en muchos problemas también interesa conocer una magnitud asociada a cada conexión. Esa magnitud recibe el nombre de **peso**.

El peso puede representar distintas cosas según el problema:

- distancia entre ciudades,
- tiempo de viaje,
- costo económico,
- consumo de combustible,
- ancho de banda de una red.

*Recordar: "Cuando un grafo almacena este tipo de información se denomina **grafo valorado** o **grafo ponderado**."*

Matriz de pesos



En este ejemplo:

Villa Mercedes —— 35 —— Justo Daract
Villa Mercedes —— 35 —— Fraga
Villa Mercedes ——100 —— San Luis
Merlo ——180 —— Villa Mercedes
Merlo ——200 —— San Luis
San Luis —— 65 —— Fraga

Los números representan la distancia entre las ciudades en kilómetros.

Por ejemplo:

- Villa Mercedes y Justo Daract están separadas por 35 km.
- Villa Mercedes y San Luis están separadas por 100 km.
- San Luis y Fraga están separadas por 65 km.

Construcción de la matriz de pesos

tenemos este orden : Justo Daract - Merlo - Villa Mercedes - San Luis - Fraga
Como el grafo es no dirigido, la matriz será simétrica.

P =

	JD	M	VM	SL	F
JD	0	0	35	0	0
M	0	0	180	200	0
VM	35	180	0	100	35
SL	0	200	100	0	65
F	0	0	35	65	0

Matriz de adyacencia vs Matriz de pesos

La diferencia principal entre ambas matrices está en la información que almacenan.

Matriz de adyacencia

Responde la pregunta: ¿Existe conexión?

Matriz de pesos

Responde la pregunta: ¿Cuánto cuesta recorrer la conexión?

Limitaciones de la matriz de adyacencia

La matriz de adyacencia proporciona una representación simple y eficiente para consultar si existe una conexión entre dos vértices. Sin embargo, no siempre resulta la mejor alternativa.

Recordemos que para un grafo con **n vértices** es necesario almacenar una matriz de:

$$n \times n$$

Esto implica que la memoria reservada depende exclusivamente de la cantidad de vértices y no de la cantidad real de aristas existentes.

Como consecuencia, un grafo con pocas conexiones ocupa exactamente la misma memoria que otro con muchísimas conexiones.

Grafos dispersos

Un grafo disperso es aquel en el que la cantidad de aristas es pequeña respecto de la cantidad máxima posible.

En este tipo de grafos la matriz de adyacencia suele contener una enorme cantidad de ceros.

Por ejemplo: si modelamos las correlatividades de una carrera universitaria, una materia puede depender de dos o tres materias anteriores, pero difícilmente dependa de cincuenta.

Lo mismo ocurre en un organigrama empresarial, donde cada empleado reporta a muy pocas personas.

En estos casos la matriz desperdicia una gran cantidad de espacio almacenando relaciones inexistentes.

Surgimiento de las listas de adyacencia

Para resolver el problema del desperdicio de memoria aparece una representación diferente denominada **lista de adyacencia**.

La idea es almacenar únicamente las conexiones que realmente existen.

En lugar de reservar espacio para todas las relaciones posibles entre vértices, cada vértice mantiene una lista con sus vecinos.

De esta manera, la memoria utilizada crece de acuerdo con la cantidad real de aristas presentes en el grafo.

Esta representación resulta especialmente adecuada para grafos dispersos.

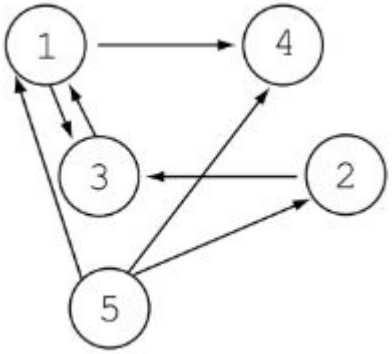
Estructura conceptual

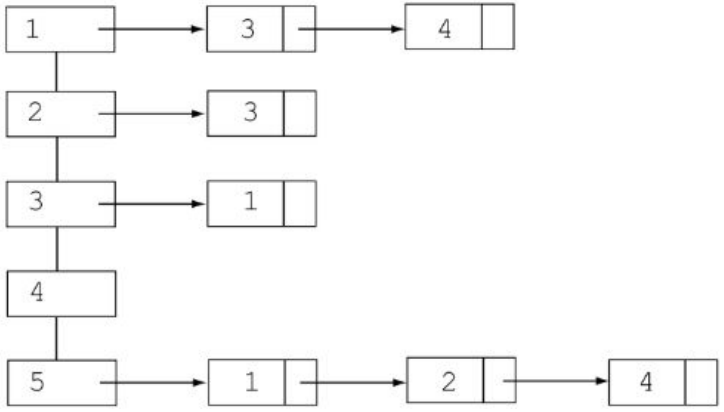
Una lista de adyacencia puede interpretarse como una tabla directorio.

Cada posición de la tabla corresponde a un vértice.

Asociada a cada vértice existe una lista enlazada que contiene todos los vértices adyacentes.

La estructura permite conocer rápidamente cuáles son los destinos alcanzables desde un determinado nodo.





Análisis de complejidad espacial

Una de las razones más importantes para utilizar listas de adyacencia es el ahorro de memoria.

La matriz de adyacencia requiere almacenar: $\mathbf{V^2}$ elementos.

Por esta razón su complejidad espacial es: $\mathbf{O(V^2)}$. independientemente de la cantidad de aristas.

Las listas de adyacencia almacenan:

- todos los vértices,
- únicamente las aristas existentes.

Su complejidad espacial es: $\mathbf{O(V + E)}$

- V representa la cantidad de vértices.
- E representa la cantidad de aristas.

Comparación de operaciones

La elección entre matriz y lista no depende únicamente de la memoria.

También depende del costo de las operaciones que se realizarán sobre el grafo.

Por ejemplo, verificar si existe una arista entre dos vértices es extremadamente rápido utilizando una matriz.

Basta acceder a una posición determinada:

matriz[A][B]

La complejidad es: **O(1)**

En una lista de adyacencia, en cambio, es necesario recorrer la lista del vértice origen buscando el vértice destino.

La complejidad depende del número de vecinos que posea dicho vértice.

¿Por qué las listas son ideales para recorridos?

Cuando se ejecutan algoritmos como DFS *(Depth First Search)* o BFS *(Breadth First Search)*, normalmente interesa conocer los vecinos de un vértice.

Con una matriz es necesario inspeccionar toda una fila para descubrir cuáles son los vértices adyacentes.

Con listas de adyacencia los vecinos ya están almacenados explícitamente.

Por esta razón, la mayoría de los algoritmos modernos sobre grafos dispersos suelen implementarse utilizando listas de adyacencia.

Criterio práctico de elección

En términos generales:

- Si el grafo es denso, la matriz de adyacencia suele ser una alternativa razonable.
- Si el grafo es disperso, las listas de adyacencia suelen resultar más eficientes.
- Si la operación predominante consiste en consultar rápidamente la existencia de una arista, la matriz ofrece ventajas.
- Si predominan los recorridos y exploraciones del grafo, las listas suelen ser la mejor opción.

La decisión final siempre depende del problema que se desea resolver y de las operaciones que tendrán mayor frecuencia de ejecución.

Qué significa recorrer un grafo?

Hasta ahora hemos estudiado cómo representar un grafo y cómo almacenar sus vértices y aristas.

Sin embargo, en la mayoría de los problemas no alcanza con almacenar la información. También es necesario explorar el grafo para obtener datos sobre él.

Recorrer un grafo consiste en visitar sistemáticamente todos los vértices alcanzables desde un vértice inicial.

Durante el recorrido pueden realizarse distintas tareas:

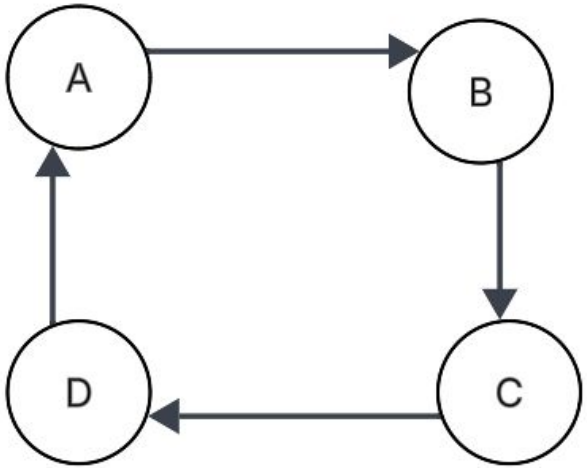
- Buscar un vértice determinado.
- Verificar si existe un camino entre dos vértices.
- Detectar ciclos.
- Encontrar componentes conexas.

Idea general de un recorrido

Recorrer un grafo consiste en visitar todos los vértices alcanzables desde un vértice inicial.

El recorrido comienza en un nodo de partida y, a partir de él, se exploran sus conexiones.

Posteriormente se exploran las conexiones de los vértices encontrados, y así sucesivamente hasta que no queden vértices por descubrir.



Esto es fundamental porque muchos grafos contienen ciclos.

Sin un mecanismo de marcado podrían producirse recorridos infinitos.

Cómo se organiza un recorrido?

Cuando se visita un vértice aparecen nuevos vértices que deberán analizarse más adelante.

Por lo tanto, el algoritmo necesita una estructura auxiliar para almacenar temporalmente los vértices pendientes de procesar.

Esa estructura contiene los vértices descubiertos pero aún no explorados.

Dependiendo de cómo se gestione esta colección de vértices pendientes aparecen dos estrategias clásicas de recorrido.

Dos estrategias fundamentales

Existen dos formas tradicionales de recorrer un grafo:

Recorrido en anchura (Breadth First Search – BFS)

Explora primero los vértices más cercanos al origen.

Utiliza una cola como estructura auxiliar.

Recorrido en profundidad (Depth First Search – DFS)

Avanza lo más profundo posible antes de retroceder.

Utiliza una pila o recursión.

Ambos algoritmos visitan los mismos vértices alcanzables.

La diferencia está en el orden en que se exploran.

Idea intuitiva del recorrido en anchura (BFS)

El recorrido en anchura (*Breadth First Search - BFS*) explora un grafo de manera progresiva, avanzando por niveles de cercanía respecto del vértice de partida.

La característica principal de este método es que visita primero todos los vértices más próximos al origen antes de avanzar hacia vértices más alejados.

Si imaginamos que cada arista representa un "salto", BFS recorre primero los vértices que se encuentran a un salto del origen, luego los que están a dos saltos, después los que están a tres, y así sucesivamente.

Por qué BFS utiliza una cola?

La cola sigue la política:

Primero en entrar
Primero en salir
(FIFO)

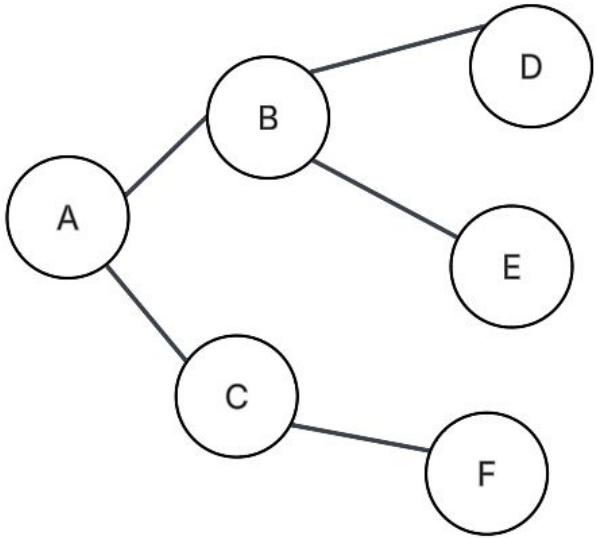
Esto garantiza que los primeros vértices descubiertos sean también los primeros en procesarse.

Como consecuencia, todos los vértices de un nivel se exploran antes de pasar al siguiente.

Esta propiedad es precisamente la que produce el recorrido por capas o niveles.

Ejemplo conceptual de BFS

Consideremos el siguiente grafo



Comenzando en A:

Nivel 0:

A

Nivel 1:

B C

Nivel 2:

D E F

Orden de visita posible:

A, B, C, D, E, F

Implementación eficiente con listas de adyacencia

Durante un recorrido es necesario consultar constantemente los vértices adyacentes de cada nodo.

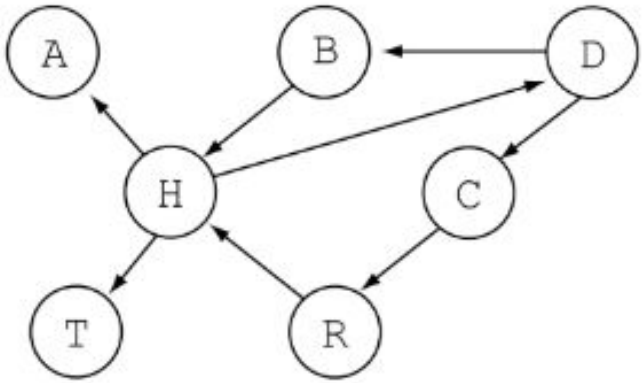
Por esta razón, BFS suele implementarse utilizando listas de adyacencia.

Con esta representación:

- cada vértice se visita una sola vez;
- cada arista se analiza una sola vez.

La complejidad temporal es: **O(V + E)**

Ejemplo de Recorrido



El recorrido en anchura, a partir del nodo D, del grafo de la figura ha accedido a todos los
nodos del grafo; se puede afirmar que todos sus vértices son alcanzables desde el vértice D.

Recorramos el Grafo a partir del Nodo D

COLA	NODO PROC.
D	
BC	D
CH	B
HR	C
RAT	H
AT	R
T	A
COLA VACÍA	T

Recorrido en profundidad (DFS)

El recorrido en profundidad (Depth First Search, DFS) tiene el mismo objetivo que el recorrido en anchura:

> visitar todos los vértices alcanzables desde un vértice de partida.

La diferencia entre ambos algoritmos no está en los vértices que visitan, sino en el orden en que los exploran.

Mientras BFS explora primero los vértices más cercanos al origen, DFS intenta avanzar lo más lejos posible antes de retroceder.

Por qué DFS utiliza una pila?

La estructura auxiliar utilizada es una pila.

Recordemos que una pila sigue la política:

LIFO
Last In First Out

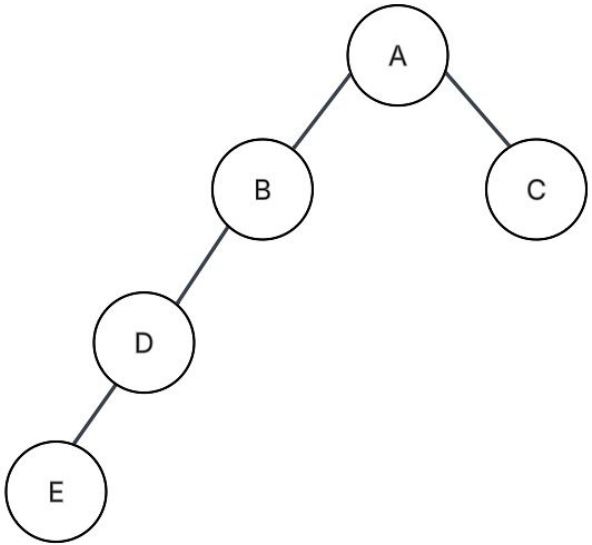
Último en entrar, primero en salir.

Esto provoca que el vértice descubierto más recientemente sea el siguiente en procesarse.

Como consecuencia, el algoritmo continúa profundizando antes de explorar otros caminos.

Ejemplo conceptual

Consideremos el siguiente grafo:



Si comenzamos desde A:

DFS avanzará siguiendo un único camino mientras sea posible.

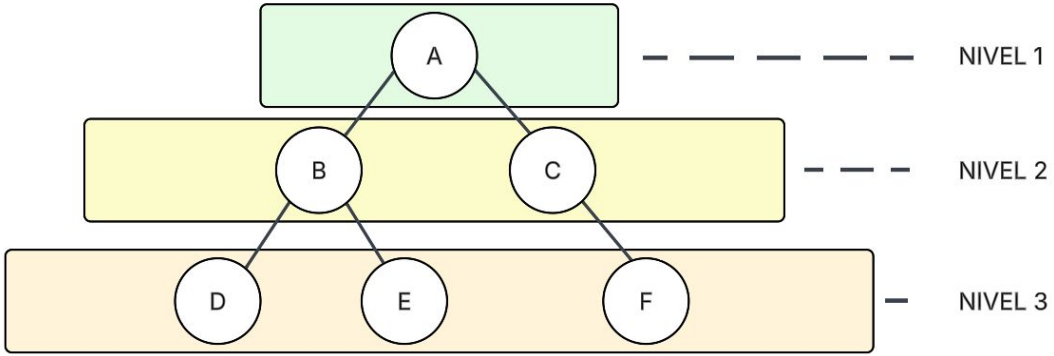
Posible recorrido:

A → B → D → E

Al llegar a E no existen nuevos vértices para visitar.

Entonces el algoritmo retrocede.

Comparación visual BFS vs DFS

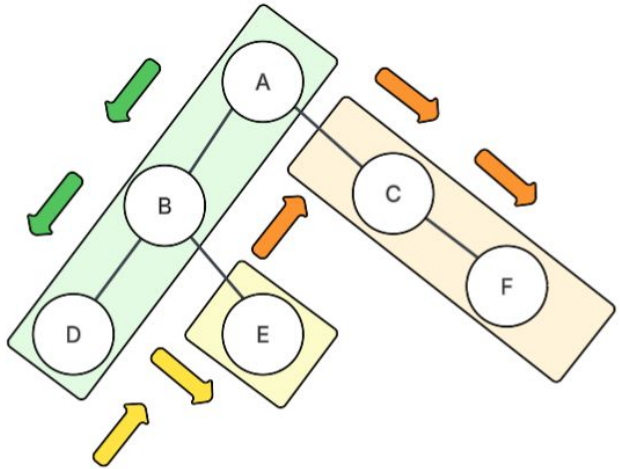


Recorrido BFS:

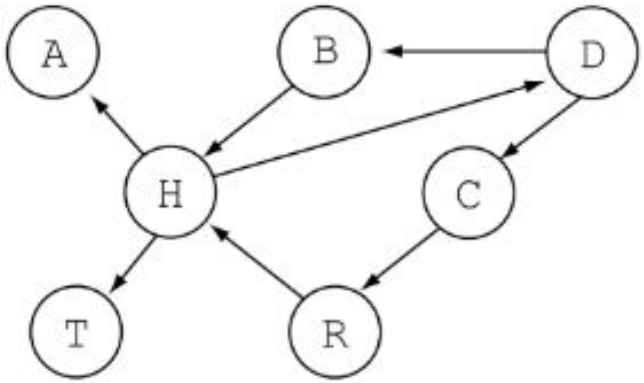
A, B, C, D, E, F

Recorrido DFS:

A, B, D, E, C, F



Ejemplo de Recorrido



Recorramos el Grafo a partir del Nodo D

PILA	NODO PROC.
D	
BC	D
BR	C
BH	R
BAT	H
BA	T
B	A
PILA VACÍA	B

Todos están conectados?

Hasta ahora aprendimos:

Qué es un grafo - Cómo representarlo - Cómo recorrerlo

Pero:

¿Qué tan conectado está el grafo?

Por ejemplo:

1. Desde una ciudad puedo llegar a todas las demás?
2. Desde un servidor puedo comunicarme con toda la red?
3. Desde una tarea puedo alcanzar todas las dependencias?

Muchas veces el problema no es encontrar un camino.

El problema es saber si **todo el sistema está conectado**.

Qué es una componente conexa?

Una componente conexa es una "isla" dentro del grafo.

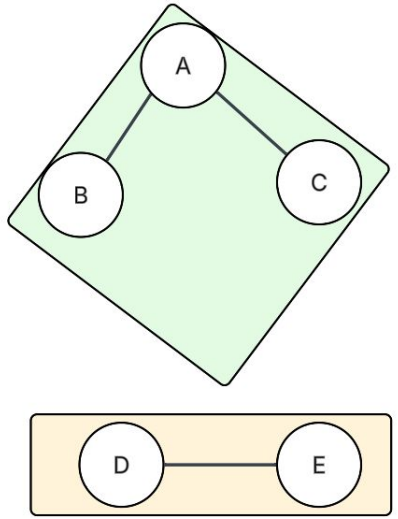
Dentro de esa isla todos los vértices están conectados.

Pero no existe conexión con otras islas.

Componentes:

{ A, B, C }

{ D, E }



Conectividad en grafos dirigidos

En un grafo no dirigido la conectividad resulta relativamente sencilla de analizar, ya que las aristas pueden recorrerse en ambos sentidos. Si existe un camino entre dos vértices, automáticamente existe también el camino inverso.

Sin embargo, en un grafo dirigido la situación cambia significativamente debido a que las aristas poseen una dirección determinada.

Esto implica que puede existir un camino desde un vértice A hacia un vértice B, pero no necesariamente desde B hacia A.

Por esta razón, el concepto de conectividad utilizado en grafos no dirigidos no resulta suficiente para describir el comportamiento de los grafos dirigidos.

Para estos casos se introduce el concepto de **conectividad fuerte**.

Grafos fuertemente conexos

Un grafo dirigido se denomina **fuertemente conexo** cuando para cualquier par de vértices u y v se cumplen simultáneamente las siguientes condiciones:

- Existe un camino desde u hasta v.
- Existe un camino desde v hasta u.

Dicho de otra manera, todos los vértices son mutuamente alcanzables.

Esta propiedad garantiza que el grafo forme una única estructura de comunicación donde cualquier vértice puede intercambiar información con cualquier otro siguiendo las direcciones permitidas.

Los grafos fuertemente conexos aparecen frecuentemente en:

- protocolos de comunicación,
- redes distribuidas.

Componentes fuertemente conexas

No todos los grafos dirigidos son fuertemente conexos.

Sin embargo, dentro de un mismo grafo pueden existir subconjuntos de vértices que sí satisfacen esta propiedad.

Estos subconjuntos reciben el nombre de **componentes fuertemente conexas**.

Una componente fuertemente conexa es un conjunto máximo de vértices donde todos los vértices pueden alcanzarse mutuamente.

La palabra "máximo" es importante porque indica que no puede agregarse ningún otro vértice al conjunto sin perder la propiedad de conectividad fuerte.

Estrategia para encontrar componentes fuertes

La determinación de componentes fuertemente conexas se basa en una idea sencilla:

Para que un vértice pertenezca a la misma componente fuerte que otro deben cumplirse dos condiciones:

1. Debo poder llegar desde el vértice inicial al vértice analizado.
2. Debo poder regresar desde ese vértice al vértice inicial.

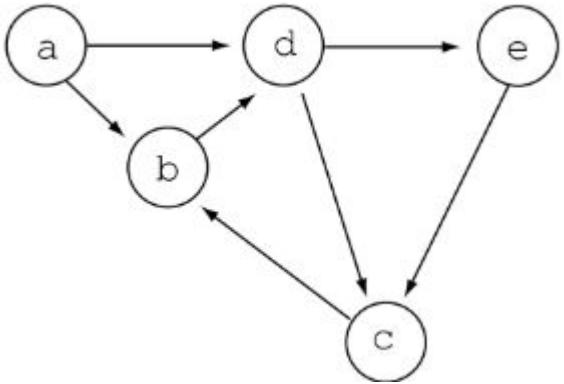
La primera condición se verifica recorriendo el grafo original.

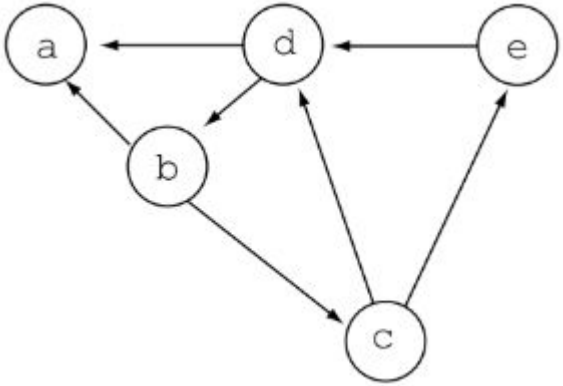
La segunda condición requiere determinar qué vértices pueden alcanzar al vértice de partida.

Para ello se utiliza el grafo inverso.

El grafo inverso

El grafo inverso se obtiene invirtiendo la dirección de todas las aristas.





Ventaja

En lugar de buscar todos los caminos posibles hacia un vértice, basta con realizar un recorrido DFS o BFS sobre el grafo invertido.

Justificación del algoritmo

Al realizar DFS sobre el grafo original obtenemos:

D(d) -> conjunto de vértices descendientes. -> todos los vértices alcanzables desde d.

Posteriormente se realiza DFS sobre el grafo inverso obteniendo:

A(d) -> conjunto de vértices ascendientes -> todos los vértices capaces de alcanzar a d en el grafo original.

La intersección:

$D(d) \cap A(d)$

contiene exactamente los vértices que cumplen ambas condiciones.

Por lo tanto, los vértices pertenecientes a esa intersección forman una componente fuertemente conexa.

El problema que queremos resolver

Hasta ahora aprendimos a determinar si dos vértices están unidos directamente mediante una arista.

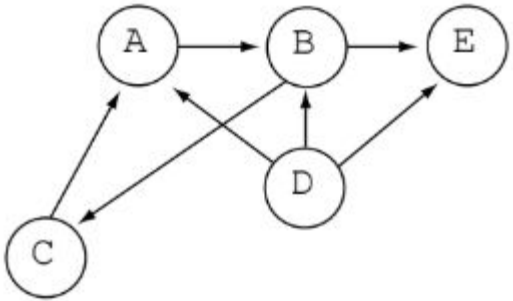
Por ejemplo, observando una matriz de adyacencia podemos responder fácilmente:

¿Existe una conexión directa entre A y B?

Sin embargo, muchas veces la pregunta importante es otra:

¿Existe algún camino entre A y E?

aunque para llegar sea necesario atravesar varios vértices intermedios.



Aunque no existe una arista directa:

A → E

sí existe un camino:

A → B → E

De la matriz de adyacencia a la matriz de caminos

La matriz de adyacencia únicamente almacena información sobre conexiones directas.

Por ejemplo: A[i,j] = 1,   indica que existe una arista desde el vértice i hasta el vértice j.

Pero: A[i,j] = 0 , no significa necesariamente que no exista forma de llegar.

Puede existir un camino utilizando vértices intermedios.

La matriz indica: $A \rightarrow E = 0$

porque no existe arista directa.

Por eso necesitamos una estructura que represente caminos y no solamente adyacencias.

La idea detrás de A^2

Supongamos que queremos descubrir caminos de longitud 2.

Un camino de longitud 2 tiene exactamente:

3 vértices
2 aristas

Ejemplo: $A \rightarrow B \rightarrow E$

Para que exista este camino deben cumplirse simultáneamente dos condiciones:

$A \rightarrow B$ y $B \rightarrow E$

Es decir:

existe un vértice intermedio que conecta origen y destino.

Interpretación de A^2

El producto matricial:

$A \times A = A^2$

permite encontrar automáticamente todos esos caminos de longitud 2.

Cada elemento de la matriz:

$A^2(i,j)$

representa caminos de longitud 2 entre los vértices i y j.

Si: $A^2(i,j) = 0$; no existe ningún camino de longitud 2.

Si: $A^2(i,j) = 1$; existe al menos un camino de longitud 2.

Si: $A^2(i,j) = 3$; existen tres caminos diferentes de longitud 2.

Por qué funciona?

La multiplicación matricial analiza todos los posibles vértices intermedios.

Cuando calcula:

$A^2(i,j)$

está preguntando:

> ¿Existe algún vértice k tal que pueda ir de i a k y luego de k a j?

Por eso:

A^2 encuentra caminos de longitud exactamente 2.

Caminos de longitud 3

Si A^2 descubre caminos de longitud 2:

A^2 = caminos de longitud 2

entonces:

$A^2 \times A = A^3$

descubre caminos de longitud 3.

Ahora buscamos recorridos formados por:

4 vértices
3 aristas

Generalización

El razonamiento puede extenderse indefinidamente.

A^1 → caminos de longitud 1

A^2 → caminos de longitud 2

A^3 → caminos de longitud 3

...

Ak → caminos de longitud k

Por lo tanto:

La potencia k de la matriz de adyacencia permite conocer caminos de longitud k.

Esta propiedad es una de las aplicaciones más importantes del álgebra matricial en teoría de grafos.

Qué información aporta Ak?

El elemento:

Ak (i,j)

no solamente indica si existe un camino.

También indica:

Cuántos caminos distintos de longitud k existen entre i y j.

Ejemplo:

Ak(i,j) = 4

significa que existen cuatro recorridos diferentes de exactamente k aristas entre ambos vértices.

Complejidad computacional

Aunque el método es matemáticamente elegante, presenta un problema importante.

La multiplicación de matrices requiere:

$O(n^3)$ operaciones.

Si además necesitamos calcular:

A^2, A^3, A^4 ... Ak

Complejidad aproximada:

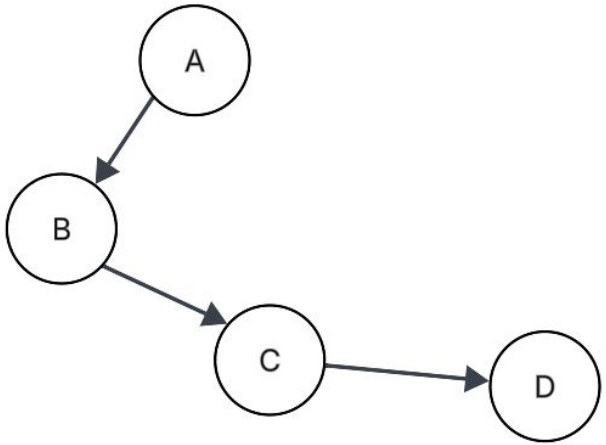
$O(k \cdot n^3)$

Por qué necesitamos el cierre transitivo?

Hasta ahora vimos que la matriz de adyacencia nos permite saber si existe una conexión directa entre dos vértices.

Sin embargo, en muchos problemas la pregunta importante no es si existe una arista directa, sino si es posible llegar de un vértice a otro utilizando vértices intermedios.

Por ejemplo, en el siguiente grafo:



No existe una conexión directa entre A y D.

Sin embargo, sí existe un camino que permite llegar desde A hasta D pasando por B y C.

Qué es el cierre transitivo?

El cierre transitivo de un grafo representa todas las relaciones de alcanzabilidad entre sus vértices.

Permite determinar si un vértice puede alcanzar a otro independientemente de la cantidad de aristas o vértices intermedios que deba recorrer.

No interesa cómo es el camino ni cuántos pasos tiene.

Únicamente interesa saber si existe alguna forma de llegar.

Interpretación práctica

El cierre transitivo puede entenderse como una vista completa de todas las conexiones posibles dentro del grafo.

Mientras la matriz de adyacencia muestra únicamente los enlaces directos, el cierre transitivo muestra todos los vértices que pueden alcanzarse desde cada nodo.

Cierre transitivo y algoritmo de Warshall

Hasta este punto vimos que las potencias de la matriz de adyacencia permiten descubrir caminos de distintas longitudes:

A → caminos de longitud 1
A^2 → caminos de longitud 2
A^3 → caminos de longitud 3
...
An → caminos de longitud n

Por lo tanto, conceptualmente, el cierre transitivo puede obtenerse combinando todas esas matrices:

$T = A + A^2 + A^3 + \cdots + An$

Cierre transitivo y algoritmo de Warshall

Cuál es el problema?

Calcular todas las potencias de la matriz resulta costoso para grafos grandes.

Por esa razón se utiliza el **algoritmo de Warshall**, que obtiene el cierre transitivo sin necesidad de calcular A^2, A^3, A^4, etc.

Idea de Warshall

Si un vértice **i** puede llegar a un vértice **k**, y además **k** puede llegar a **j**, entonces también existe un camino entre **i** y **j**.